



CSI142 Object-Oriented Programming

Week 1, Lecture 3: Knowledge Check, switch, do-while, and OOP Preview

Friday, 30 Jan 2026 • 08:00-09:00 • Room 252-008 • Semester II 2025/26



Today's plan

CSI142 • Sem II 2025/26

- Quick knowledge check (Java basics)
- switch: clean multi-way decisions
- do-while: input validation and menu loops
- Bridge to Week 2 topics
- Forward look: classes and objects



Answer on paper or in your notes:

- 1) Which loop executes at least once before checking the condition?
- 2) Explain the difference between `=` and `==`.
- 3) How do you correctly compare two strings in Java?
- 4) After using `nextInt()`, why can `nextLine()` read an empty string?



```
int x = 5;
while (x > 0) {
    if (x % 2 == 0) {
        System.out.print(x + " ");
    }
    x--;
}
```

- What does it print?
- How many times does the loop run?
- What happens if we remove `x--` ?



```
switch (day) {  
    case 1:  
        System.out.println("Mon");  
        break;  
    case 2:  
        System.out.println("Tue");  
        break;  
    default:  
        System.out.println("Invalid");  
}
```

- Good for menus and fixed choices
- break prevents fall-through
- default handles unexpected values



```
System.out.println("1) Deposit");
System.out.println("2) Withdraw");
System.out.println("3) Balance");
int choice = in.nextInt();

switch (choice) {
    case 1: /* deposit */ break;
    case 2: /* withdraw */ break;
    case 3: /* show balance */ break;
    default: System.out.println("Choose 1-3");
}
```

- This pattern appears often in labs
- We will combine switch + loops to build menu programs



do-while loop: always run once

CSI142 • Sem II 2025/26

```
int n;  
do {  
    System.out.print("Enter a positive number: ");  
    n = in.nextInt();  
} while (n <= 0);
```

- Body runs at least once
- Perfect for input validation
- Often combined with switch for menus



```
int choice;
do {
    System.out.println("1) Add  2) Subtract  0) Exit");
    choice = in.nextInt();

    switch (choice) {
        case 1: /* ... */ break;
        case 2: /* ... */ break;
        case 0: System.out.println("Bye!"); break;
        default: System.out.println("Invalid choice");
    }
} while (choice != 0);
```

- Common pattern: loop until exit option
- Remember to update the loop variable (choice)



- Week 2 topics: switch, operator precedence, do-while (deeper)
- Arrays and ArrayList: storing many values, processing with loops
- Command-line arguments: passing info into `main(String[] args)`
- Practical theme: building small utilities that grow into OOP designs



Class: BankAccount

Fields (data):

- ownerName : String
- balance : double

Methods (behaviour):

- deposit(amount)
- withdraw(amount)
- getBalance()



Objects (instances)

acc1: BankAccount

Same structure, different data

acc2: BankAccount

Same structure, different data

acc3: BankAccount

Same structure, different data



- If you struggled on the knowledge check: revisit CSI141 notes on loops and I/O this weekend
- Practise building a small menu program using switch + do-while
- Come to lab with questions, and bring your code attempts
- Next week: deeper control structures and arrays, then we move into classes